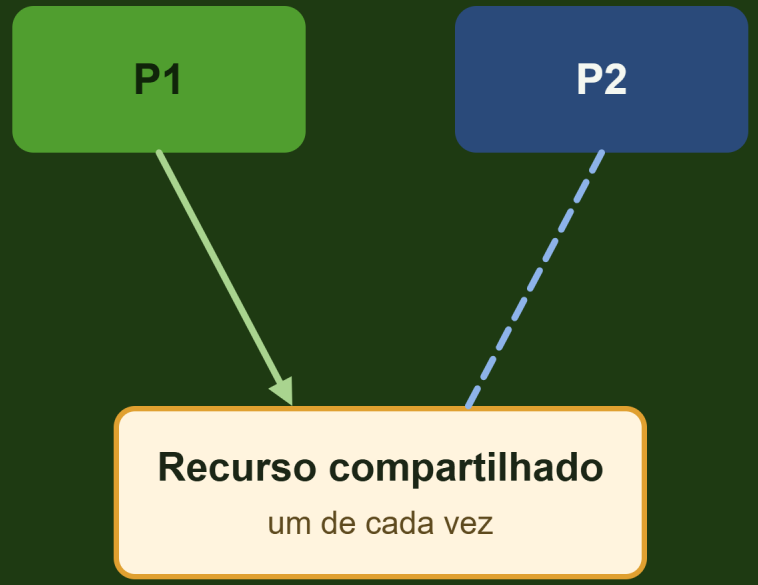
ARQUITETURA DE SISTEMAS OPERACIONAIS · CAPÍTULO 7

Sincronização e comunicação entre processos

Como processos que compartilham recursos trabalham juntos sem se atrapalhar

Prof. Christiano Rossini · IFAL Campus Maceió



CAP. 7 · ROTEIRO

A sequência do capítulo

7.1
Introdução

7.2
Aplicações concorrentes

7.3
Especificação de concorrência

7.4
Problemas de compartilhamento

7.5
Exclusão mútua

7.6
Soluções de hardware

7.7
Soluções de software

7.8
Sincronização condicional

7.9
Semáforos

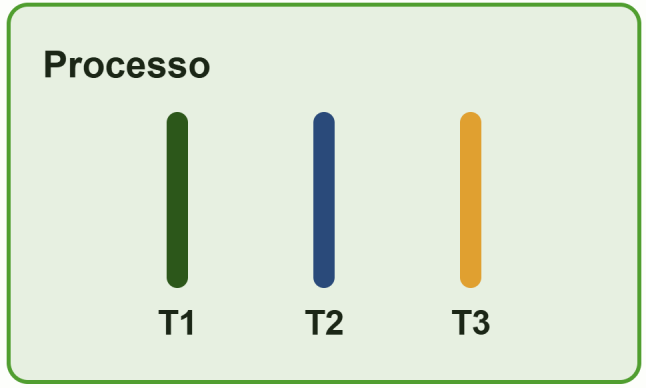
7.10
Monitores

7.11
Troca de mensagens

7.12
Deadlock

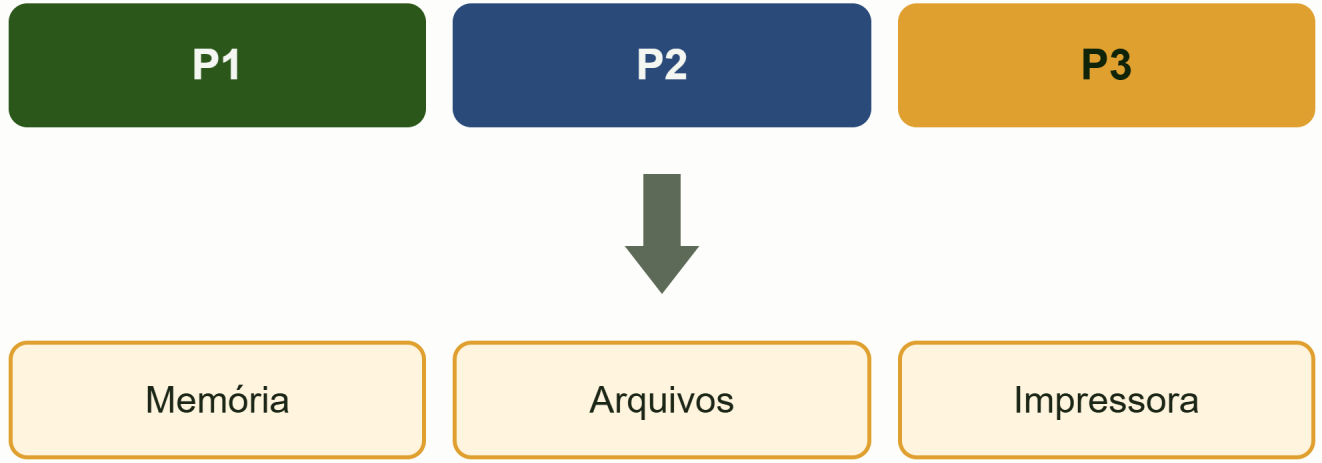
Onde paramos

CAPÍTULOS 5 E 6



Um processo pode ter vários threads, e vários processos executam ao mesmo tempo.

CAPÍTULO 7



Todos usam os **mesmos recursos**.

O que pode dar errado quando dois usam a mesma coisa ao mesmo tempo?

CAP. 7 · CONTEXTUALIZAÇÃO

Concorrência está em todo lugar

Dois caixas eletrônicos

Saque e depósito na mesma conta, ao mesmo tempo.

disputa: o saldo

Último assento do voo

Duas pessoas compram pelo site no mesmo segundo.

disputa: o assento

Laboratório com uma impressora

Dez computadores mandam imprimir juntos.

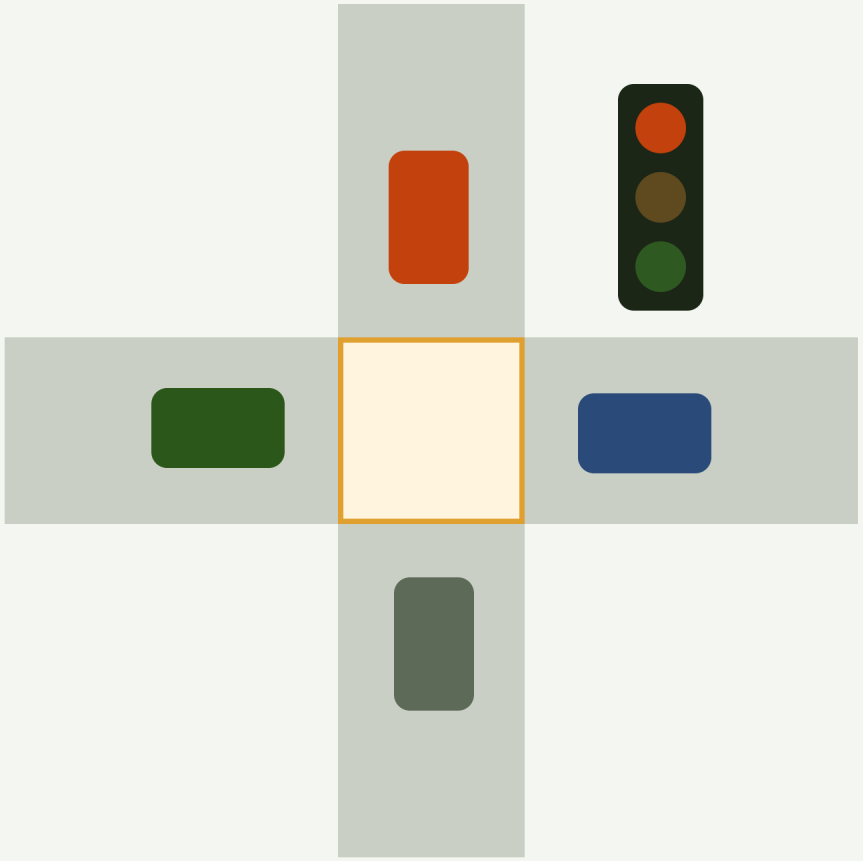
disputa: a impressora

Editor de texto

Um thread salva o arquivo enquanto você digita.

disputa: o arquivo

O trânsito como modelo



o cruzamento só comporta um carro por vez

No trânsito	No sistema operacional
cruzamento	região crítica
um carro de cada vez	exclusão mútua
semáforo de trânsito	semáforo (Dijkstra)
carro esperando o sinal abrir	processo em espera
carro que nunca consegue passar	starvation
quatro carros travados no meio	deadlock

Quando dá errado de verdade

MARS PATHFINDER · 1997

Em Marte, a sonda começou a se reiniciar sozinha

A causa: tarefas com prioridades diferentes disputando um recurso protegido por um mutex. Uma tarefa importante ficava esperando tempo demais, e o sistema se reiniciava por segurança.

A correção foi enviada da Terra, a milhões de quilômetros.

Mais perto de nós

- o mesmo ingresso vendido duas vezes
- saldo de conta errado
- arquivo corrompido
- programa que “trava” sem motivo aparente

Erros de sincronização são **raros**, **difíceis de reproduzir** e podem **custar caro**.

Competir e cooperar

Competição

Processos que nem se conhecem disputam o mesmo recurso.

Quem usa agora?

→ exclusão mútua

Cooperação

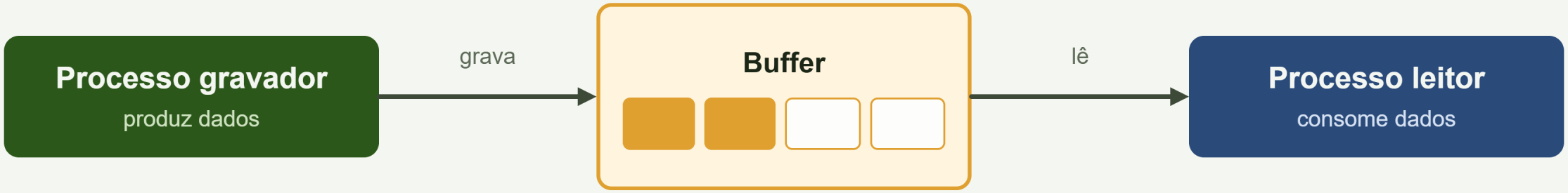
Processos que trabalham juntos precisam trocar dados.

Quando posso continuar?

→ sincronização e comunicação

condição de corrida → exclusão mútua → sincronização condicional → semáforos, monitores, mensagens → deadlock

Processos que cooperam precisam se coordenar



Comunicação

Os dados passam de um processo para o outro.

Sincronização

O leitor não lê um buffer vazio; o gravador não grava num buffer cheio.

Duas formas de os processos se comunicarem

Numa **aplicação concorrente**, vários processos ou threads executam ao mesmo tempo e trabalham juntos para resolver um problema.

Memória compartilhada



Os processos leem e escrevem nas mesmas variáveis. Rápido, mas exige cuidado com o acesso simultâneo.

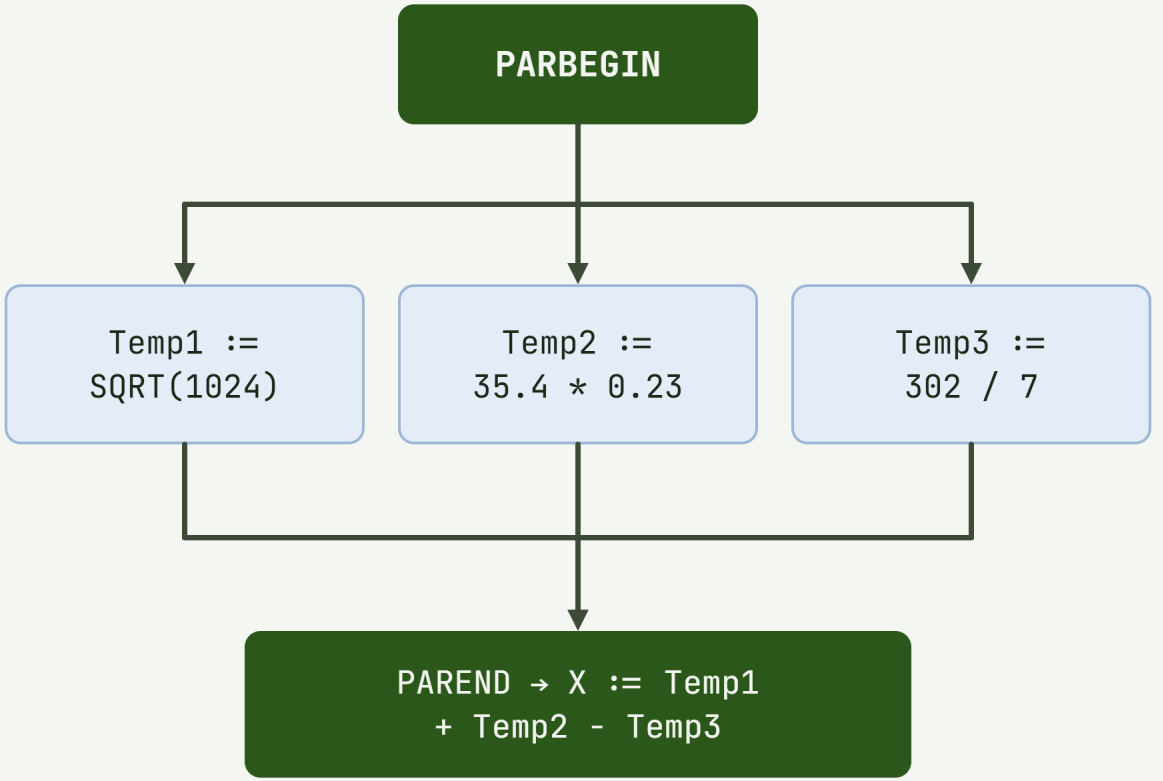
Troca de mensagens



Um processo envia, o outro recebe, com o SO no meio. Serve até para processos em computadores diferentes.

Dizendo ao programa o que pode rodar junto

```
PROGRAM Expressao;
VAR X, Temp1, Temp2, Temp3 : REAL;
BEGIN
PARBEGIN
Temp1 := SQRT (1024);
Temp2 := 35.4 * 0.23;
Temp3 := 302 / 7;
PAREND;
X := Temp1 + Temp2 - Temp3;
WRITELN ('x = ', X);
END.
```



Dois caixas, uma conta, um saldo errado

Passo	Caixa	O que faz	Saldo no arquivo
1	Caixa 1	lê o saldo: 1000	1000
2	Caixa 2	lê o saldo: 1000	1000
3	Caixa 1	saque de 200 → calcula 800	1000
4	Caixa 2	depósito de 300 → calcula 1300	1000
5	Caixa 1	grava 800	800
6	Caixa 2	grava 1300	1300 (certo: 1100)

O saque se perdeu: o caixa 2 leu o saldo **antes** de o caixa 1 gravar.

Até X := X + 1 pode dar errado

Processo A: X := X + 1

```
LOAD x, Ra
ADD 1, Ra
STORE Ra, x
```

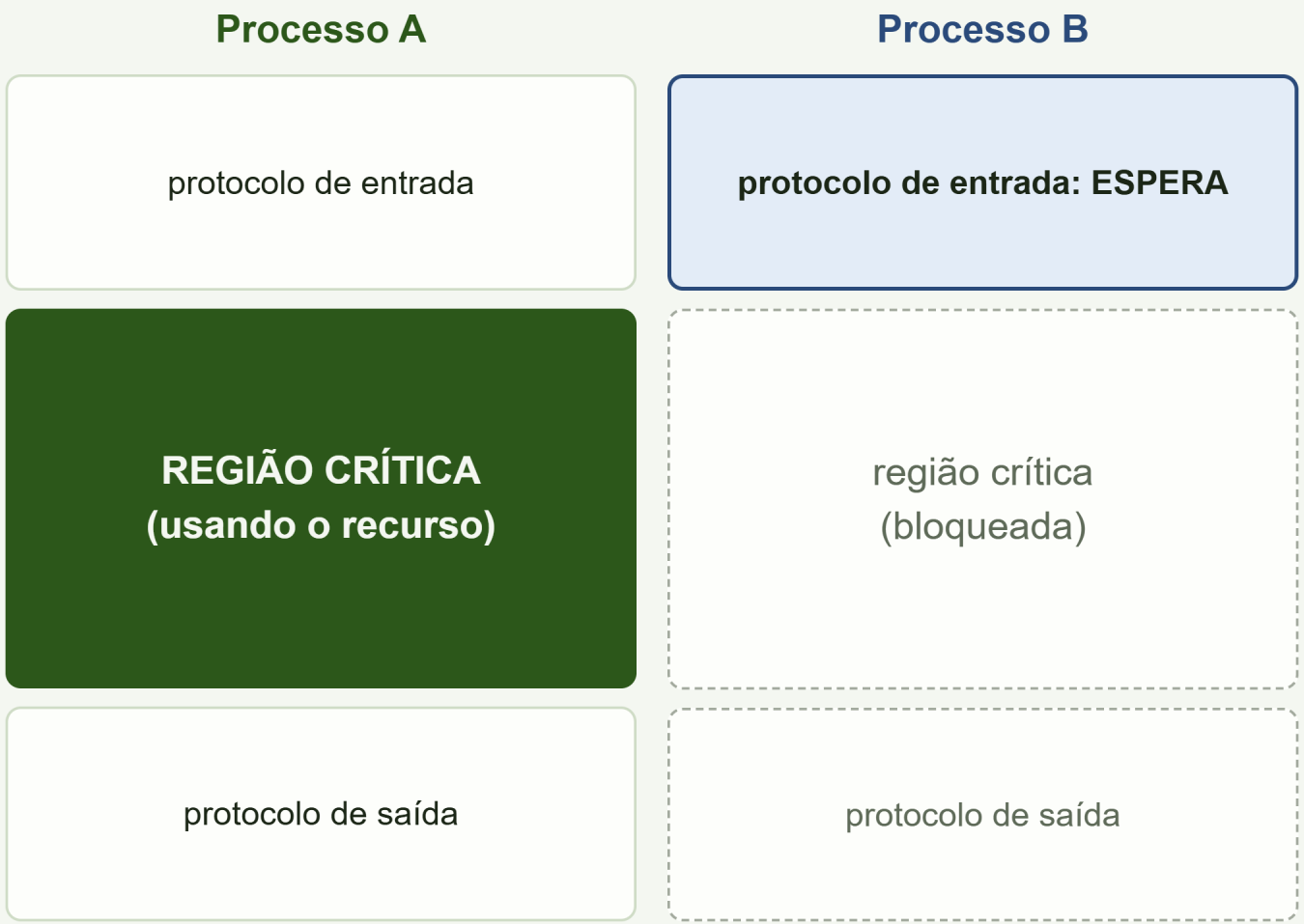
Processo B: X := X − 1

```
LOAD x, Rb
SUB 1, Rb
STORE Rb, x
```

Processo	Instrução	X	Ra	Rb
A	LOAD x, Ra	2	2	—
A	ADD 1, Ra	2	3	—
B	LOAD x, Rb	2	3	2
B	SUB 1, Rb	2	3	1
A	STORE Ra, x	3	3	1
B	STORE Rb, x	1	3	1

X começou em 2 e deveria terminar em 2. Terminou em **1**.

Região crítica: um processo de cada vez



Uma boa solução evita

- **starvation**: um processo esperando para sempre
- um processo **fora** da região crítica impedir os outros de entrar
- depender da velocidade dos processos ou do número de processadores

Duas soluções com ajuda do hardware

Desabilitar interrupções

```
Desabilita_Interrupcoes;
Regiao_Critica;
Habilita_Interrupcoes;
```

Sem interrupções, não há troca de processo durante a região crítica.

Problemas: um processo de usuário pode esquecer de reabilitar; não funciona com vários processadores.

Instrução test-and-set

```
repete Test_And_Set(bloqueio)
até conseguir;
Regiao_Critica;
bloqueio := falso;
```

Lê e altera a variável de bloqueio numa única instrução, que não pode ser interrompida.

Problemas: espera ocupada e possível starvation.

Exclusão mútua só com programação

4 tentativas
cada uma falha num requisito

→

Dekker
primeira solução correta, para 2 processos

→

Peterson (1981)
solução correta e bem mais simples

Algoritmo de Peterson · processo i (o outro é j)

```
quer[i] := verdadeiro;
vez := j;
enquanto quer[j] e vez = j: espera;
Regiao_Critica;
quer[i] := falso;
```

Ideia

Mostro que quero entrar, mas cedo a vez. Só espero se o outro também quer e a vez é dele.

Limite

Ainda há **espera ocupada**: o processo gasta UCP testando a condição.

Esperar até que uma condição seja verdadeira



Buffer **cheio**: o produtor espera até haver uma posição livre.



Buffer **vazio**: o consumidor espera até chegar um dado.

Semáforo: um contador com duas operações

Proposto por **Dijkstra (1965)**: uma variável inteira, não negativa, alterada só por DOWN e UP, que são **indivisíveis**.

DOWN (S) · também P

```
se S > 0
então S := S - 1
senão o processo
fica em ESPERA
```

UP (S) · também V

```
se há processo esperando
então ACORDA um deles
senão S := S + 1
```

Binário (mutex)

vale 0 ou 1: exclusão mútua

Contador

vale 0 a N: recursos com N unidades e sincronização condicional

Exclusão mútua com semáforo

```
Mutex := 1;
REPEAT
DOWN (Mutex);
Regiao_Critica;
UP (Mutex);
UNTIL False;
```

Processo	Operação	Mutex	Resultado
A	DOWN	1 → 0	A entra
B	DOWN	0	B vai para ESPERA
A	UP	0	acorda B, que entra
B	UP	0 → 1	região crítica livre

Produtor e consumidor com três semáforos

Vazio = N posições livres

Cheio = 0 posições ocupadas

Mutex = 1 acesso ao buffer

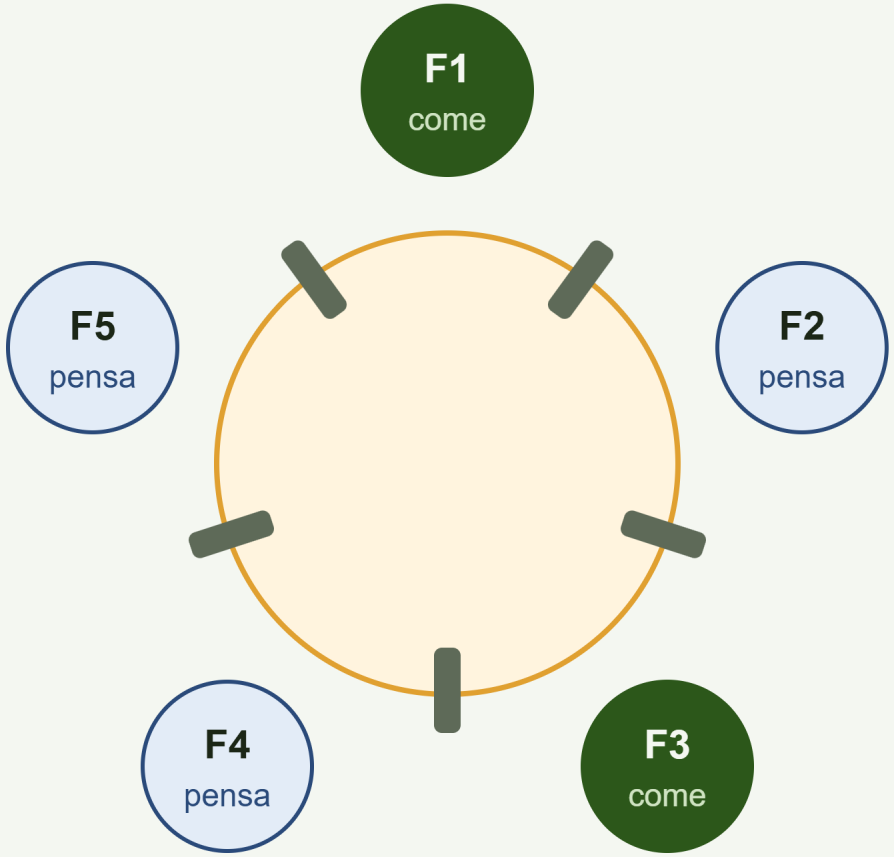
Produtor

```
REPEAT
Produz_Dado;
DOWN (Vazio);
DOWN (Mutex);
Grava_Buffer;
UP (Mutex);
UP (Cheio);
UNTIL False;
```

Consumidor

```
REPEAT
DOWN (Cheio);
DOWN (Mutex);
Le_Buffer;
UP (Mutex);
UP (Vazio);
Consome_Dado;
UNTIL False;
```

O jantar dos filósofos



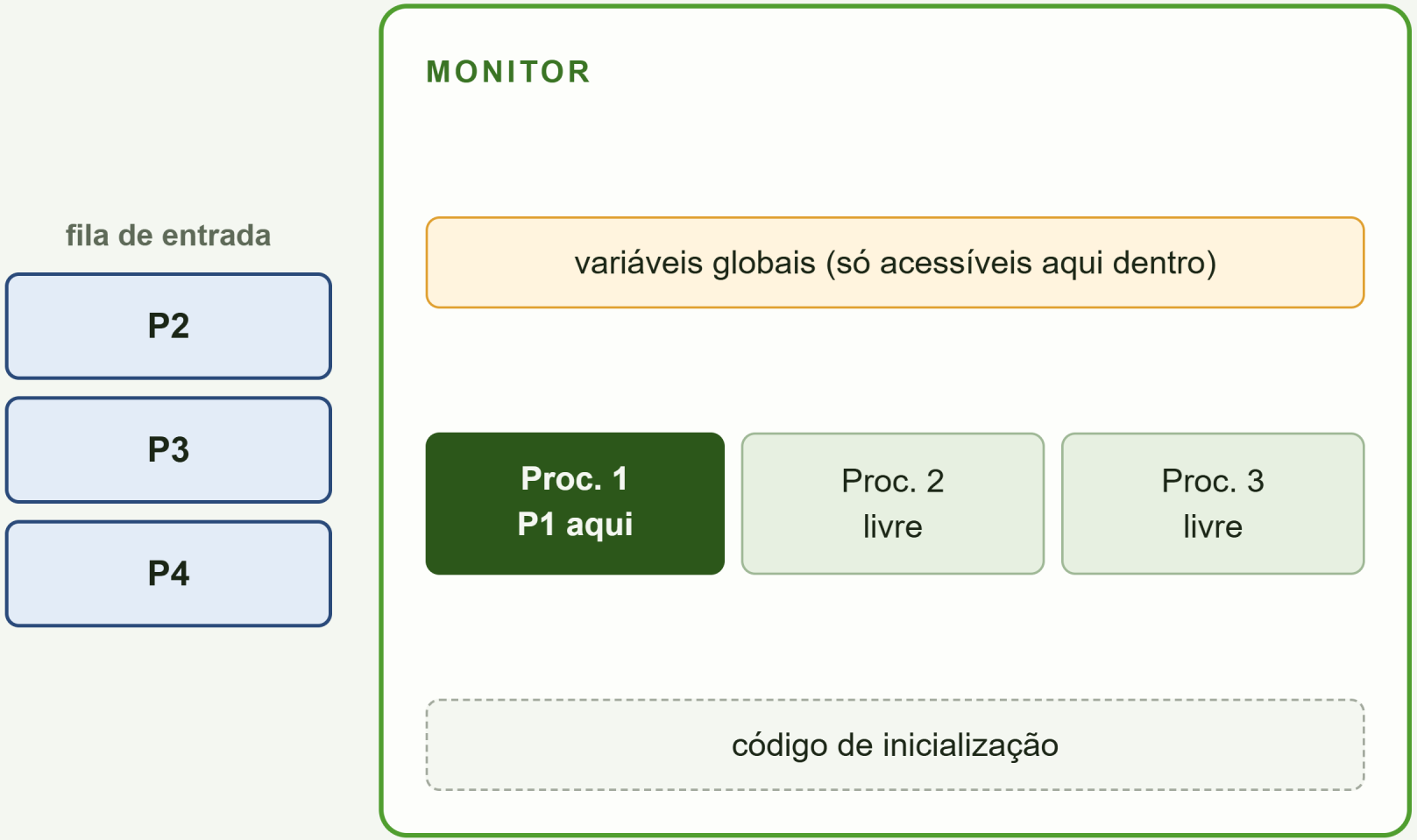
Cinco filósofos, cinco garfos. Para comer, cada um precisa dos **dois garfos** ao seu lado.

Deadlock: se todos pegarem o garfo da esquerda ao mesmo tempo, ninguém consegue o da direita.

Soluções

- no máximo quatro filósofos à mesa
- pegar os dois garfos só se ambos estiverem livres
- ímpares pegam primeiro o da esquerda; pares, o da direita

Monitor: a exclusão mútua vem pronta



Regra

Só um processo executa dentro do monitor por vez; os outros esperam na fila.

Vantagem

O compilador garante a exclusão mútua: menos erros que com DOWN e UP espalhados pelo código.

Esperar dentro do monitor: WAIT e SIGNAL

```
MONITOR Buffer;
VAR Cheio, Vazio : CONDITION;
PROCEDURE Produz;
IF Cont = TamBuf THEN WAIT (Cheio);
grava no buffer; Cont := Cont + 1;
IF Cont = 1 THEN SIGNAL (Vazio);
PROCEDURE Consome;
IF Cont = 0 THEN WAIT (Vazio);
lê do buffer; Cont := Cont - 1;
IF Cont = TamBuf - 1 THEN SIGNAL (Cheio);
END.
```

WAIT (C)

O processo espera na fila da condição C e libera o monitor para outro.

SIGNAL (C)

Acorda um processo que espera em C.

Comunicar sem compartilhar memória

```
SEND (Receptor, Mensagem)
```

```
RECEIVE (Transmissor, Mensagem)
```

Comunicação direta



Cada um nomeia o outro. Simples, mas se o nome de um processo mudar, o código do outro também muda.

Comunicação indireta



As mensagens vão para uma caixa postal (ou porta). Vários processos podem usar a mesma caixa.

Síncrona ou assíncrona?

Síncrona (rendezvous) quem envia espera o outro receber



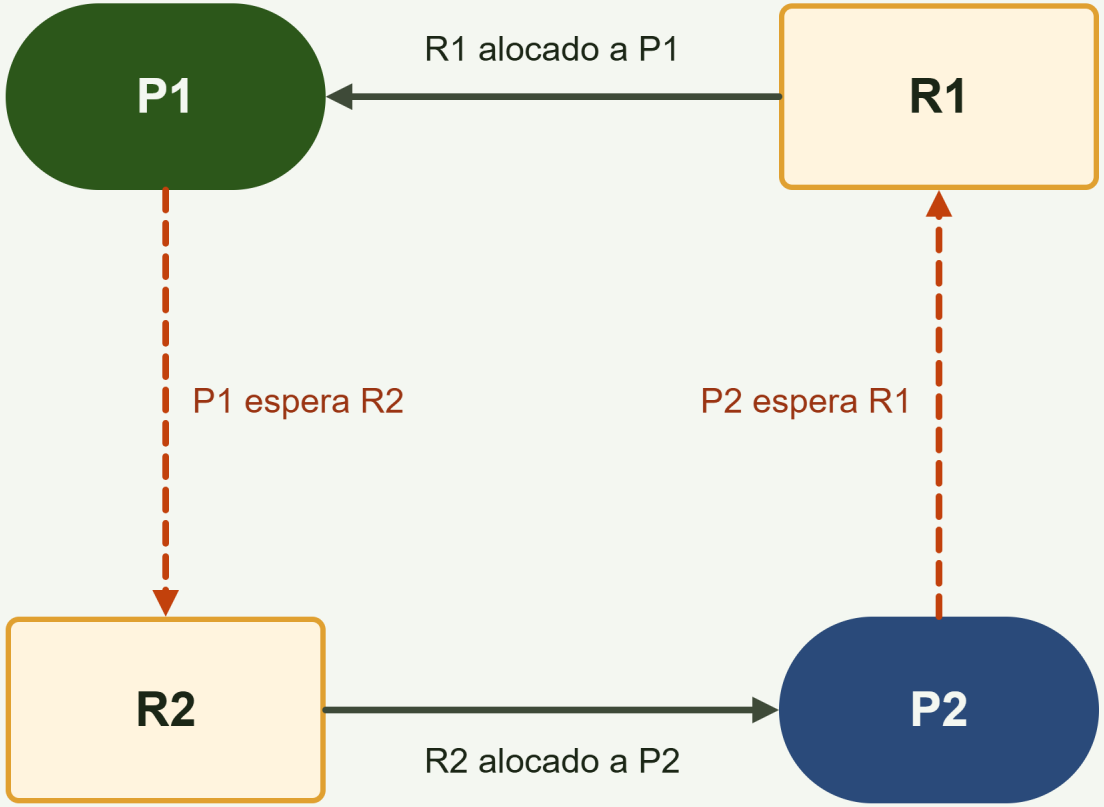
azul claro = transmissor bloqueado até o RECEIVE

Assíncrona a mensagem fica num buffer; ninguém para



mais paralelismo, mas exige buffers e controle de quando a mensagem foi lida

Deadlock: cada um espera o que o outro tem



Um processo espera por um recurso que **nunca** será liberado, porque quem o tem também está esperando.

O ciclo no grafo (P1 → R2 → P2 → R1 → P1) é o sinal de deadlock.

Seta cheia: recurso alocado. Tracejada: processo esperando.

Quatro condições, todas ao mesmo tempo

1

Exclusão mútua

Cada recurso só pode estar com um processo por vez.

2

Espera por recurso

Um processo segura recursos enquanto espera por outros.

3

Não preempção

Um recurso não pode ser tirado de um processo só porque outro o quer.

4

Espera circular

Uma cadeia em que cada processo espera um recurso do próximo.

Prevenir, detectar, corrigir

7.12.1

Prevenção

Impedir uma das quatro condições. Ex.: pedir todos os recursos de uma vez; numerar os recursos e pedir sempre em ordem.

Algoritmo do banqueiro: só concede o recurso se o sistema continuar num estado seguro.

7.12.2

Detecção

Deixar acontecer e procurar: o SO mantém estruturas com quem tem e quem espera cada recurso e busca ciclos.

Custo: a busca consome UCP; quanto mais frequente, mais cara.

7.12.3

Correção

Quebrar o ciclo: eliminar um ou mais processos envolvidos, liberando seus recursos.

Alternativa: rollback, que volta um processo a um ponto anterior sem eliminá-lo.

O capítulo em um quadro

Processos que compartilham recursos precisam de **exclusão mútua** para não se atrapalhar e de **sincronização** para trabalhar na ordem certa.

O problema

- condição de corrida
- região crítica
- starvation

Soluções

- hardware e software (espera ocupada)
- semáforos
- monitores

Comunicação

- SEND e RECEIVE
- direta ou por mailbox
- síncrona ou assíncrona

Deadlock

- quatro condições
- prevenção, detecção, correção

Para revisar

1. Por que o resultado de X := X + 1 pode sair errado com dois processos?
2. O que é região crítica e o que uma boa solução de exclusão mútua deve evitar?
3. O que é espera ocupada e como os semáforos a eliminam?
4. Qual a vantagem dos monitores em relação aos semáforos?
5. Quais as quatro condições para o deadlock? Basta impedir quantas?